



CHAPTER 5

ASYMPTOTIC NOTATIONS

Learning Outcomes

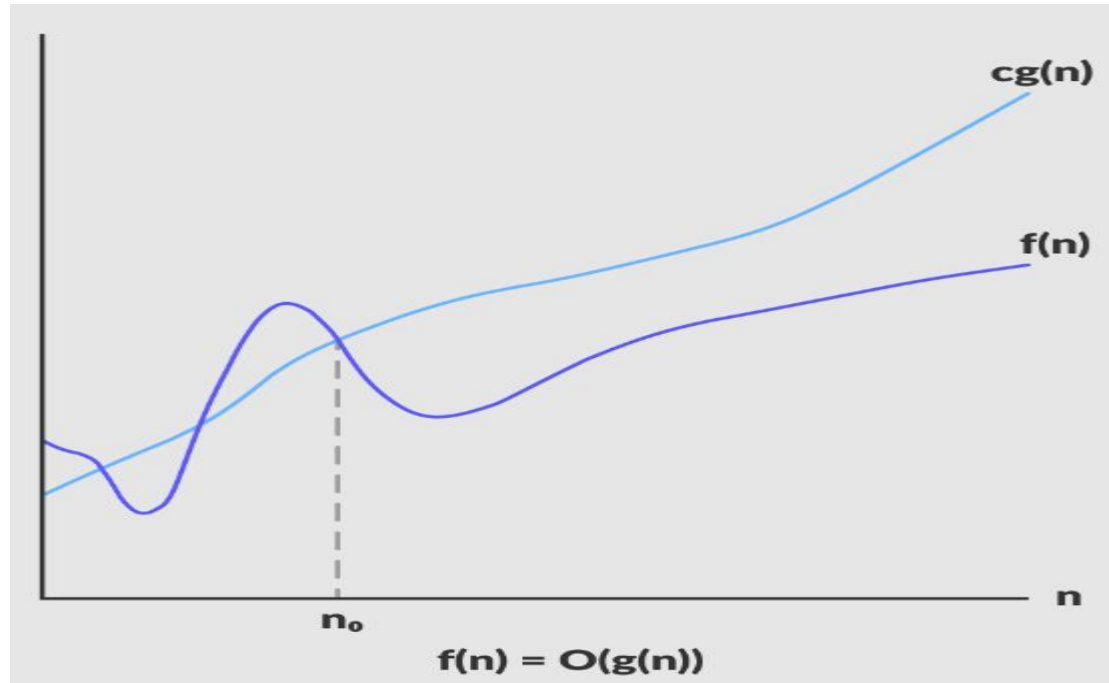
At the end of this lecture, you should be able to

1. Describe the Big O (O-notation), Big Omega(Ω -notation), and Big Theta Notations(Θ -notation).
2. Assess the efficiency of a given algorithm
3. Compare the expected execution times of two methods, given the efficiencies of their algorithms

What is Asymptotic Analysis

- ▶ In every case for a given function $f(n)$ we are trying to find another function $g(n)$ which approximates $f(n)$ at higher values of n .
- ▶ That means $g(n)$ is also a curve which approximates $f(n)$ at higher values of n .
- ▶ In mathematics we call such a curve an asymptotic curve.
- ▶ In other terms, $g(n)$ is the asymptotic curve for $f(n)$.
- ▶ For this reason, we call algorithm analysis asymptotic analysis.

Sample of graph



Asymptotic Notation

- ▶ Having the expressions for the best, average and worst cases, for all three cases we need to identify the upper and lower bounds.
- ▶ To represent these upper and lower bounds, we need some kind of syntax.
- ▶ Let us assume that the given algorithm is represented in the form of function $f(n)$.

Asymptotic Notation

- ▶ Asymptotic notation, also known as Big O notation, Big Omega notation, and Big Theta notation.
- ▶ It is a mathematical notation used in computer science and mathematics to describe the growth rate of functions and algorithms.
- ▶ It is commonly used in the context of analyzing the time complexity and space complexity of algorithms.

Algorithm Efficiency

- ▶ An algorithm's efficiency is determined by the order of growth of that algorithm.
- ▶ This order of growth, in turn, is determined by the basic operation count.
- ▶ Now, to compare, contrast and rank the order of growth of an algorithm, we use three notations O (Big Oh), Ω (Big omega), Θ (Big theta).

Definition

- ▶ **Big O (O)** notation is the **upper bound** that is the maximum time complexity of a program.
- ▶ **Big Omega notation (Ω)** is known as the **lower bound**, it is the minimum time at least it will take for the program to run
- ▶ **Big Theta notation (Θ)** is known as the tight bound that means it is **within both the upper bound and lower bound.**

O - notation

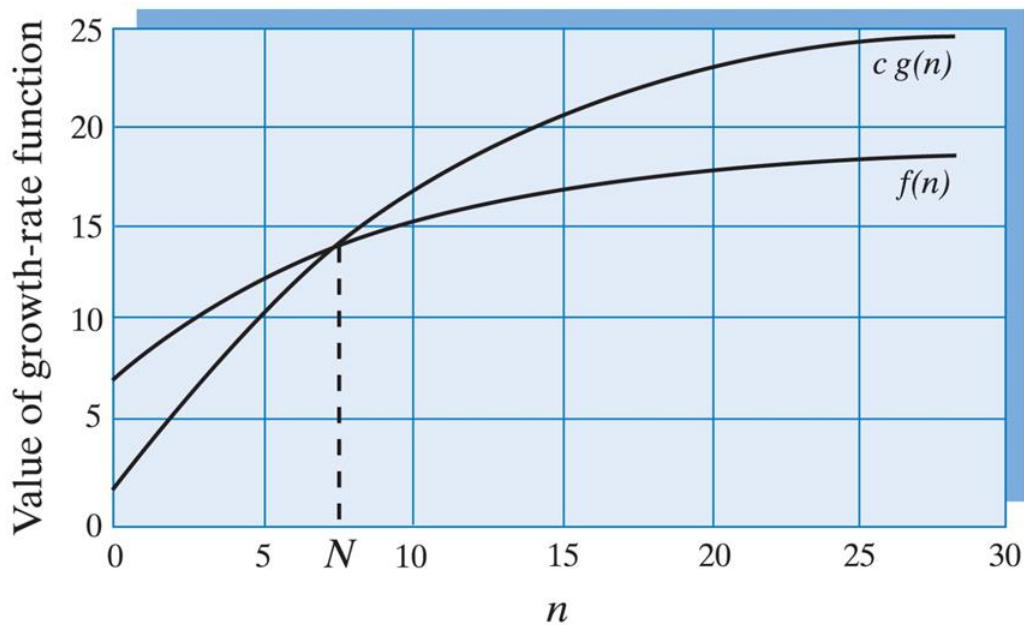
- ▶ Formal definition of Big Oh

An algorithm's time requirement $f(n)$ is of order at most $g(n)$

- ▶ $f(n) = O(g(n))$
- ▶ For a positive real number c and positive integer N exist such that

$$f(n) \leq c \cdot g(n) \text{ for all } n \geq N$$

Formalities (Optional)

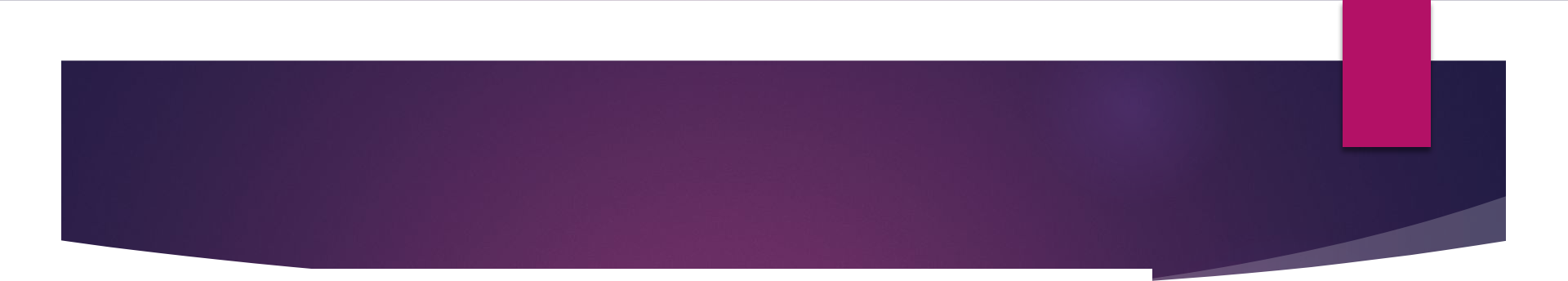


Example

- ▶ Example 1: Consider another algorithm with a running time of $2n^2 + 4n + 1$. The dominant term is n^2 , and lower-order terms and constant factors are ignored. So, the Big O notation for this algorithm would be $O(n^2)$.
- ▶ Example 2: If an algorithm has a running time of $7\log n + 5$, the dominant term is $\log n$, and constant factors are ignored. So, the Big O notation for this algorithm would be $O(\log n)$.

Example

Algorithm	Growth-rate function	Big Oh Notation
Algorithm A	$2n + 1$	$O(n)$
Algorithm B	$n^2 + n + 1$	$O(n^2)$



Algorithm A = **$2n + 1$**

= **$2n$** , ignoring smaller terms

= **n** , ignoring the coefficient 2

$\Rightarrow$ **$O(n)$**

Algorithm B = **$n^2 + n + 1$**

= **n^2** , ignoring smaller terms

$\Rightarrow$ **$O(n^2)$**

Ω - notation

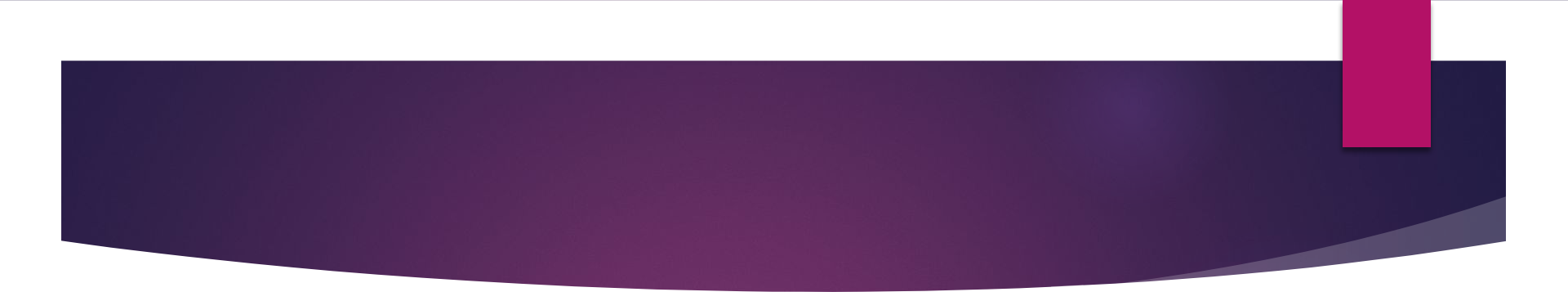
- ▶ Big Omega (Ω -notation)
- ▶ Big Omega notation represents the lower bound or best-case scenario of an algorithm's time complexity.
- ▶ It represents the minimum amount of time an algorithm takes to solve a problem as the input size approaches infinity.

Θ - notation

- ▶ This notation is an overlap of the previous two notations. $t(n)$ is said to be in $\Theta(g(n))$ if $t(n)$ is bounded both above and below by some positive constant multiple of $g(n)$ for all large values of n , if there exist some positive constants c_1 and c_2 and some non-negative integer N . It is of the form
- ▶ $c_2g(n) \leq t(n) \leq c_1g(n)$ for all $n \geq N$

Assess the efficiency of a given algorithm

- ▶ Assessing the efficiency of a given algorithm involves analyzing its time complexity and space complexity to understand how it performs as the input size grows.
- ▶ Overall, assessing the efficiency of a given algorithm involves a combination of theoretical analysis, practical benchmarking, and understanding the specific requirements and constraints of the problem at hand.

- 
- ▶ Analyze the algorithm
 - ▶ Determine the time complexity
 - ▶ Determine the space complexity
 - ▶ Consider best-case and worst-case scenarios
 - ▶ Compare with other algorithms
 - ▶ Benchmark and test
 - ▶ Optimize if necessary
 - ▶ Understand trade-offs

The 7 Most Common Functions in Algorithm Analysis

The Constant Function: $f(n) = c$

- ▶ Where c is a fixed constant (e.g., $c = 5$, $c = 210$)
- ▶ I.e., for any argument n , the constant function $f(n)$ assigns the value c .
- ▶ Therefore, it doesn't matter what the value is n is; $f(n)$ will always be equal to the constant value c .
- ▶ This function characterizes the number of steps needed to do a basic operation (e.g., adding 2 numbers, assigning a value to a variable, comparing 2 numbers)

The Logarithm Function: $f(n) = \log_b n$

- ▶ For some constant $b > 1$
- ▶ b is the base of the logarithm and for any base $b > 0$, $\log_b 1 = 0$
- ▶ $x = \log_b n$ iff $b^x = n$
- ▶ The most common base in computer science is 2 as computers store integers in binary. Therefore, the default logarithm base for us is 2, i.e. $\log n = \log_2 n$

The Linear Function: $f(n) = n$

- ▶ I.e., given an input value n , the linear function f assigns the value n itself.
- ▶ This function arises in algorithm analysis any time we have to do a single basic operation for each of n elements.
 - ▶ E.g., comparing a number x to each element of an array of size n will require n comparisons.

The n-log-n function: $f(n) = n \log n$

- ▶ This function grows a little more rapidly than the linear function and a lot less rapidly than the quadratic function.

The Quadratic Function: $f(n) = n^2$

- ▶ The main reason why the quadratic function appears in the analysis of algorithms is that there are many algorithms that have nested loops, where the inner loop performs a linear number of operations and the outer loop is performed a linear number of times.
- ▶ Thus, in such cases, the algorithm performs
- ▶ $n * n = n^2$ operations

The Cubic Function: $f(n) = n^3$

- ▶ This function appears less frequently than the constant, linear and quadratic functions.

The Exponential Function: $f(n)=b^n$

- ▶ Where b is a positive constant called the base, and the argument n is the exponent.
- ▶ In algorithm analysis, the most common base for the exponential function is $b = 2$.
- ▶ E.g., if we have a loop that starts by performing one operation and then doubles the number of operations performed with each iteration, then the number of operations performed in the n th iteration is 2^n .

Growth-Rate Functions

n	$\log(\log n)$	$\log n$	$\log^2 n$	n	$n \log n$	n^2	n^3	$2n$	$n!$
10	2	3	11	10	33	10^2	10^3	10^3	10^5
10^2	3	7	44	100	664	10^4	10^6	10^{30}	10^{94}
10^3	3	10	99	1000	9966	10^6	10^9	10^{301}	10^{1435}
10^4	4	13	177	10,000	132,877	10^8	10^{12}	10^{3010}	$10^{19,335}$
10^5	4	17	276	100,000	1,660,964	10^{10}	10^{15}	$10^{30,103}$	$10^{243,338}$
10^6	4	20	397	1,000,000	19,931,569	10^{12}	10^{18}	$10^{301,030}$	$10^{2,933,369}$

Fig. 2.4: Typical growth-rate functions evaluated at increasing values of n
 (Note: logarithms are base-2)

Picturing Efficiency

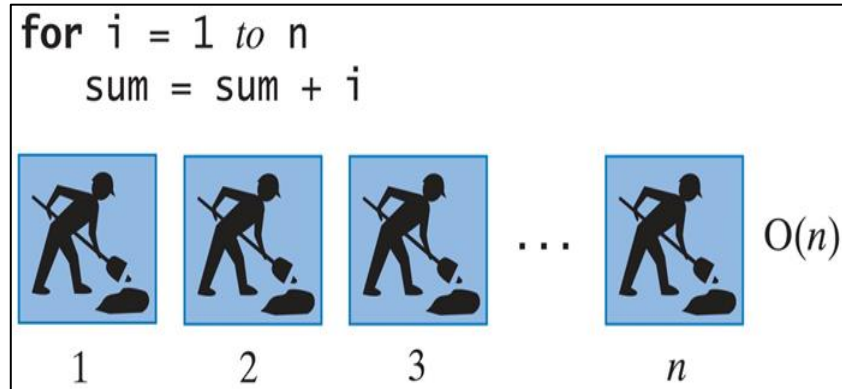


Fig. 5.1: an $O(n)$ algorithm.

Picturing Efficiency

Growth-Rate Function for Size n Problems	Growth-Rate Function for Size $2n$ Problems	Effect on Time Requirement
1	1	None
$\log n$	$1 + \log n$	Negligible
n	$2n$	Doubles
$n \log n$	$2n \log n + 2n$	Doubles and then adds $2n$
n^2	$(2n)^2$	Quadruples
n^3	$(2n)^3$	Multiplies by 8
2^n	2^{2n}	Squares

The effect of doubling the problem size
on an algorithm's time requirement.

Picturing Efficiency

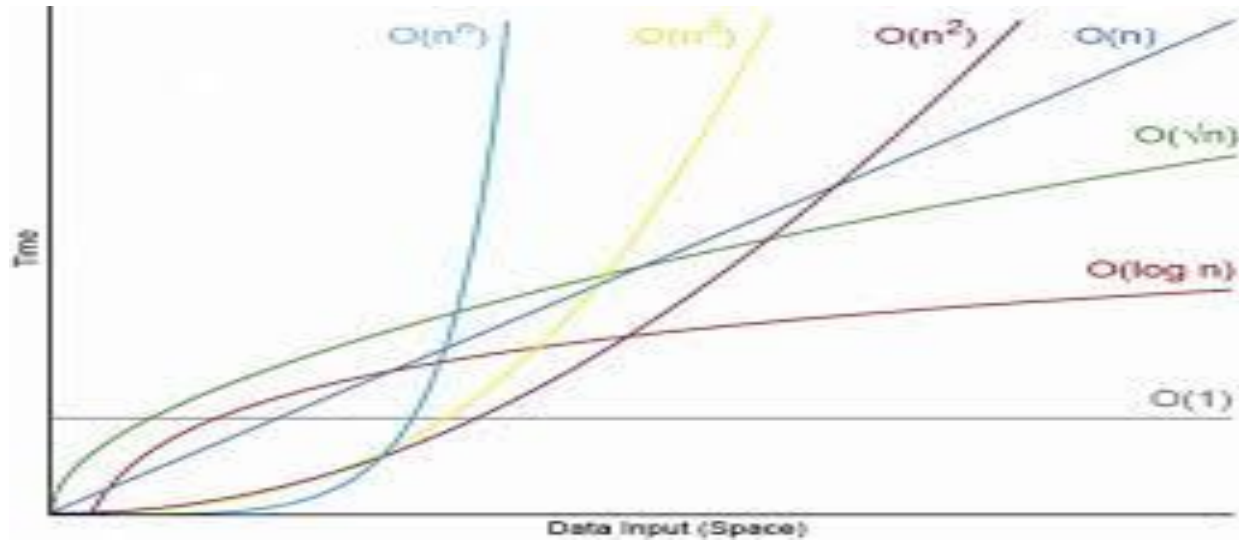
Growth-Rate Function g	$g(10^6) / 10^6$
$\log n$	0.0000199 seconds
n	1 second
$n \log n$	19.9 seconds
n^2	11.6 days
n^3	31,709.8 years
2^n	$10^{301,016}$ years

The time to process one million items by algorithms of various orders at the rate of one million operations per second.

Comments on Efficiency

- ▶ A programmer can use $O(n^2)$, $O(n^3)$ or $O(2^n)$ as long as the problem size is small

Comments on Efficiency





END OF CHAPTER 5